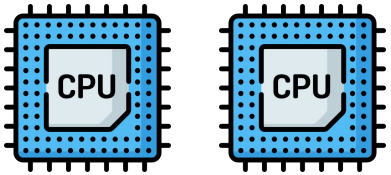
Threads and Cores

CSE 493L

Let’s write parallel code!

Our goal: +1 to an array, take advantage of multiple cores to get a speedup

Strategy: domain decomposition. Each CPU increments half of the array.



“Core” == “CPU”

Let’s write parallel code!

```
void core0_main():

for (i = 0; i < N/2; i++)

        array[i]++;
```

```
void core1_main():

for (i = N/2; i < N; i++)

        array[i]++;
```

That program is brittle…

What if the number of cores changes?

- N/2 => N/3
- Need a new main function

I've got it!

```
void core_main(int I):

start = N*I/NUM_CORES;

end = N*(I+1)/NUM_CORES;

for (i = start; i < end; i++)

        array[i]++;
```

This works, but only in domain decomposition cases…

That code is still brittle…

What if we want to run a second program on those cores?

What if we want to download and run code from online and have it just run on one core?

We need an abstraction over cores (or a “virtual core”): threads.

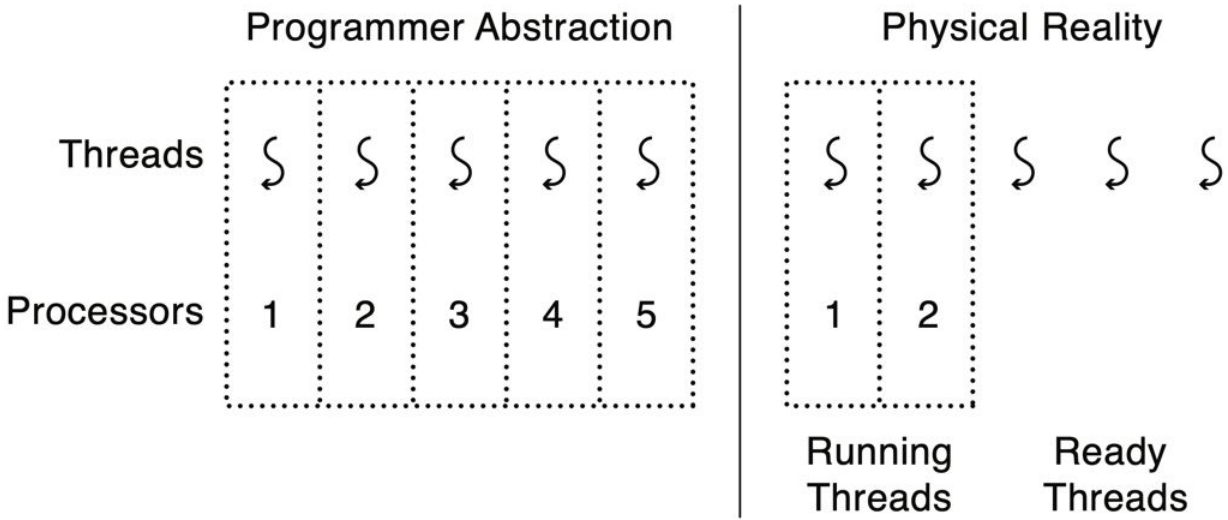
What is a Thread?

A thread is a single execution sequence, which represents a separately schedulable task

Threads are executed on cores, and are an abstraction of cores

Thread Abstraction

- Infinite number of processors
- Threads execute with variable speed
- Programs must be designed to work with any schedule



Wait, isn’t this a Coroutine?

Coroutines must *explicitly* interact with the scheduler to yield

Threads *implicitly* can be running or not at any time

In practice, a thread is a coroutine that is forced to yield every so often

- Can yield after any instruction. Much harder to reason about!

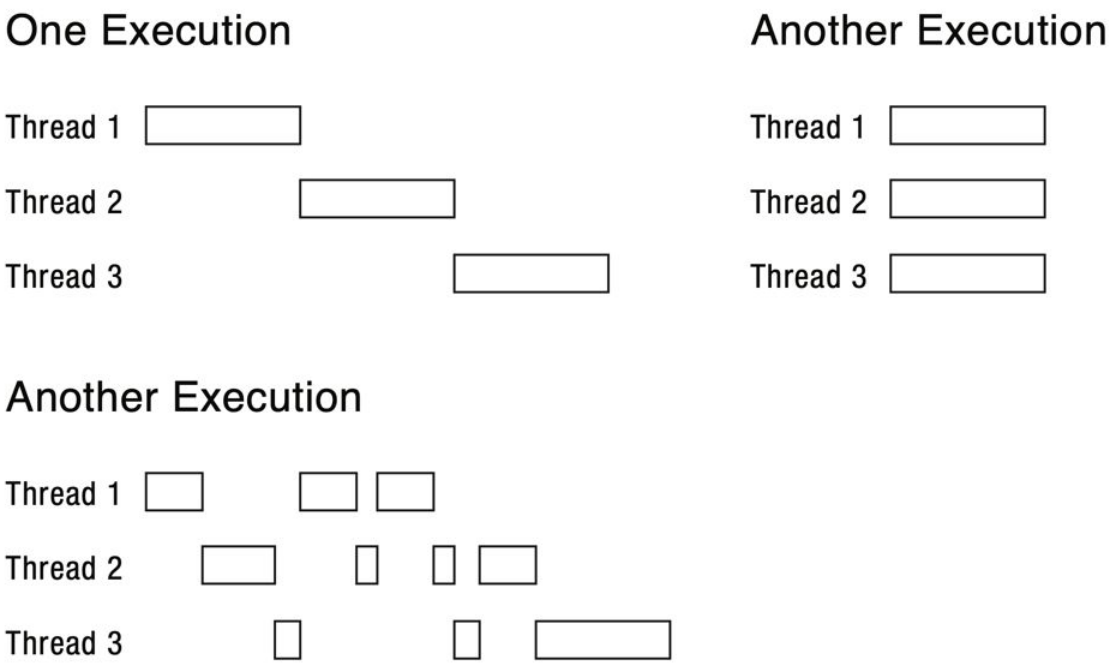
Question

Why do threads execute at a variable speed?

Programmer vs. Processor View

Programmer's View	Possible Execution #1	Possible Execution #2	Possible Execution #3
.	.	.	.
.	.	.	.
.	.	.	.
x = x + 1;	x = x + 1;	x = x + 1;	x = x + 1;
y = y + x;	y = y + x;	.....	y = y + x;
z = x + 5y;	z = x + 5y;	Thread is suspended.	.....
.	.	Other thread(s) run.	Thread is suspended.
.	.	Thread is resumed.	Other thread(s) run.
.	.	.....	Thread is resumed.
		y = y + x;	.....
		z = x + 5y;	z = x + 5y;

Possible Executions



Thread Operations

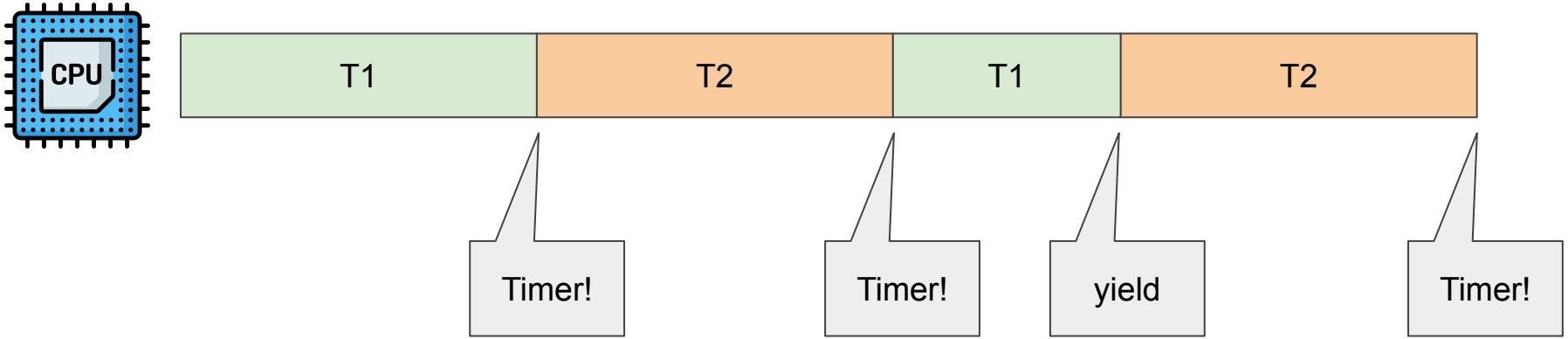
- thread_create(thread, func, args)
  - Create a new thread to run `func(args)`
- thread_yield()
  - Relinquish processor voluntarily
- thread_join(thread)
  - Wait for created thread to exit (from parent)
- thread_exit()
  - Quit thread and clean up, wake up joiners

Deja Vu?

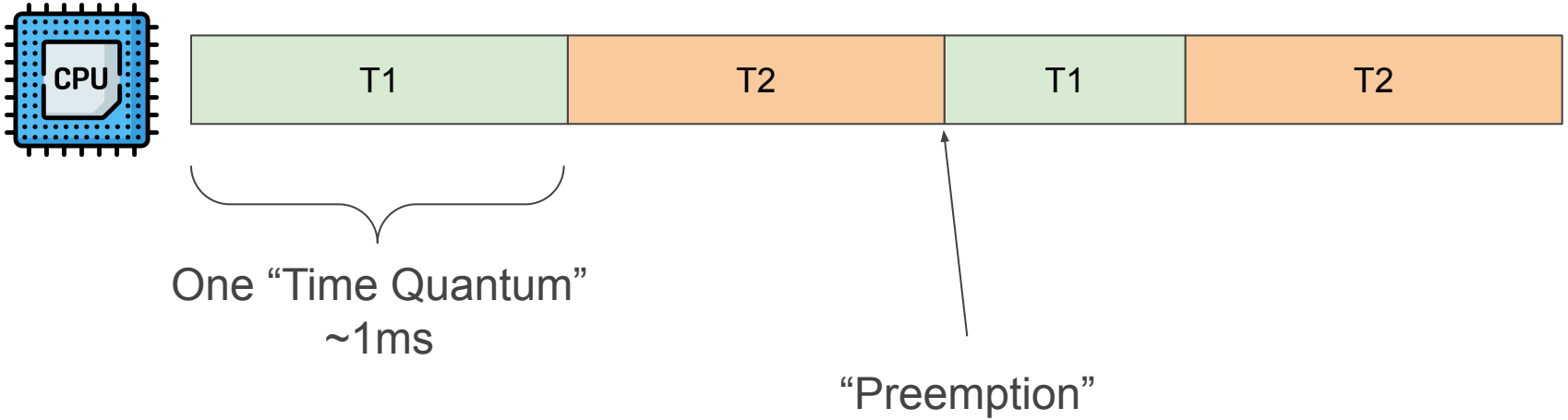
Coroutine API is the same!

Execution schedule looks different

What does this look like?



What does this look like?



How often should we preempt?

Very often: low latency for applications that want to run now, high overhead

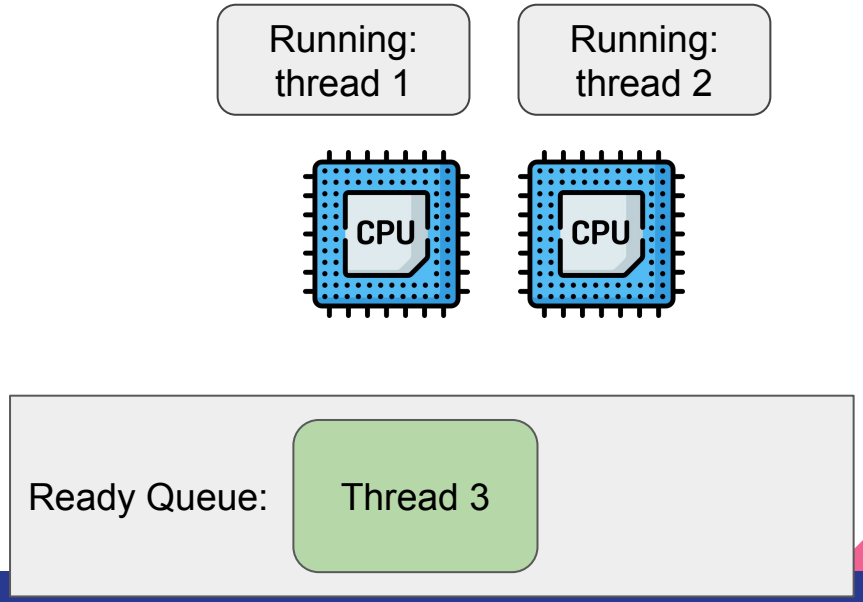
Not often: high throughput / utilization, but high latency to run unblocked threads

No right answer! It’s a tradeoff.

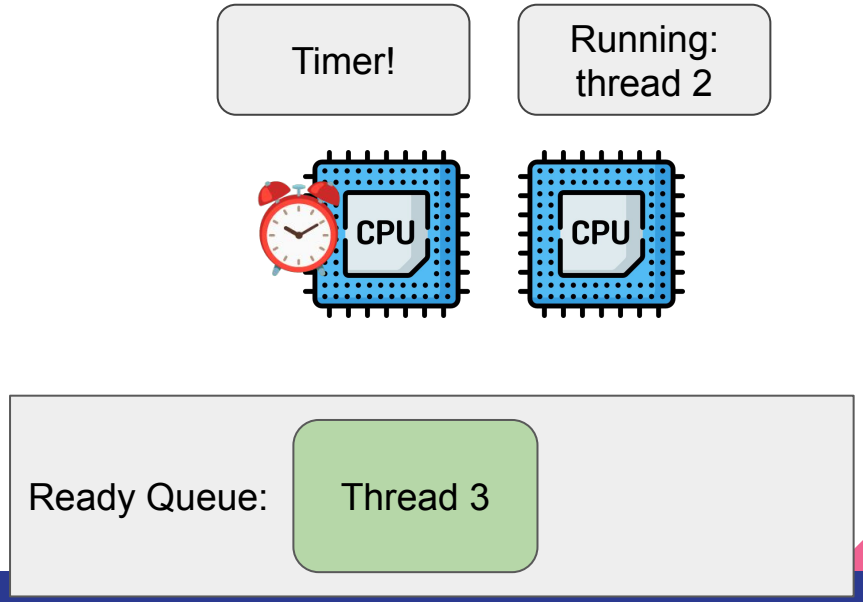
Can threads move between cores?

Yes!

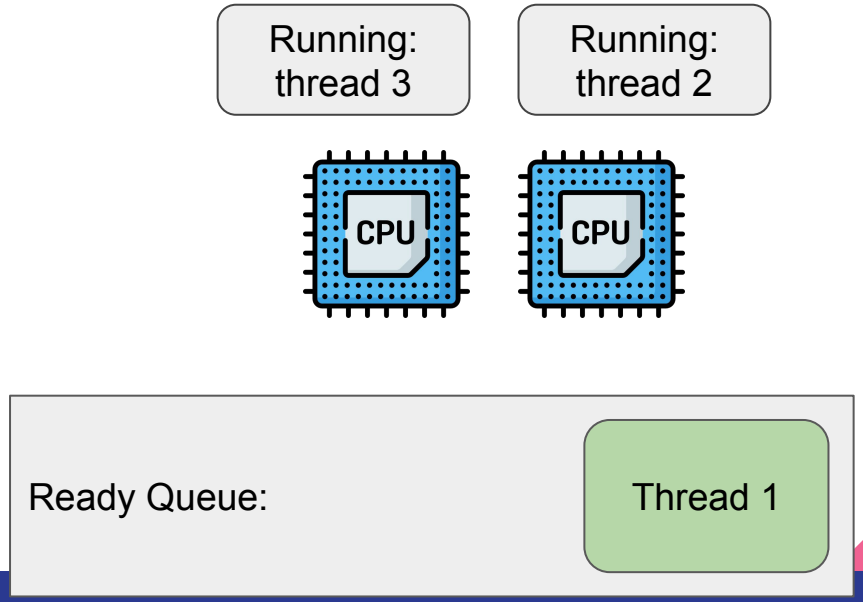
Can threads move between cores?



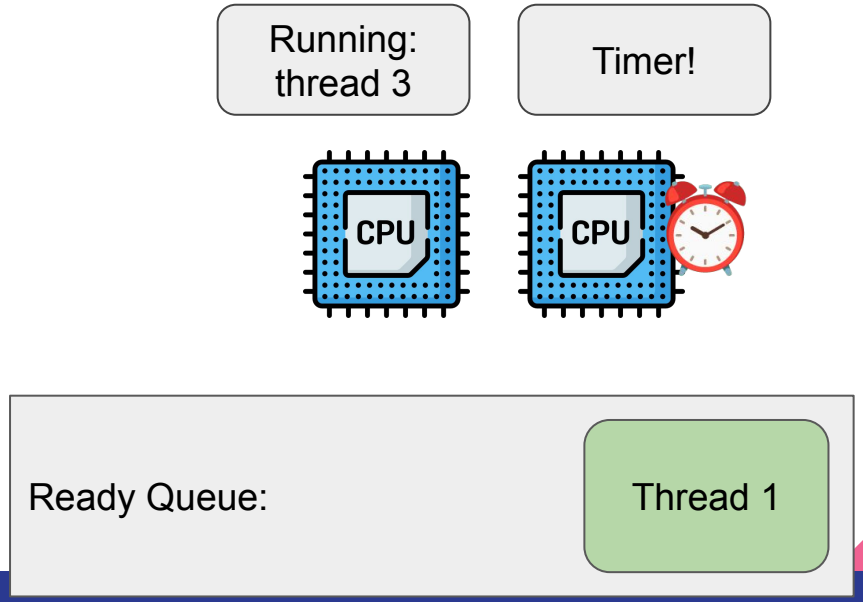
Can threads move between cores?



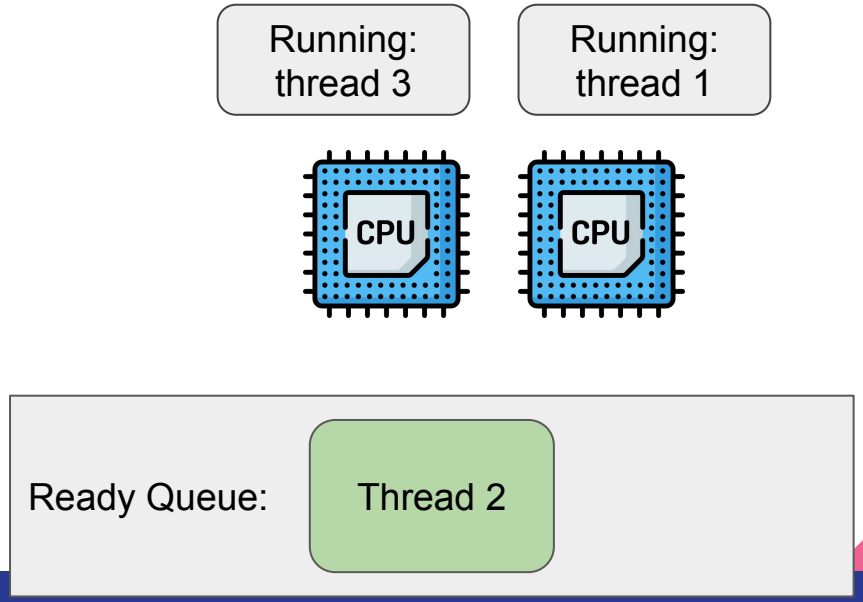
Can threads move between cores?



Can threads move between cores?



Can threads move between cores?

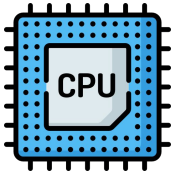


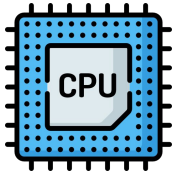
Can threads move between cores?

Is it OK for thread 1 to move cores?

Running: thread 3

Running: thread 1

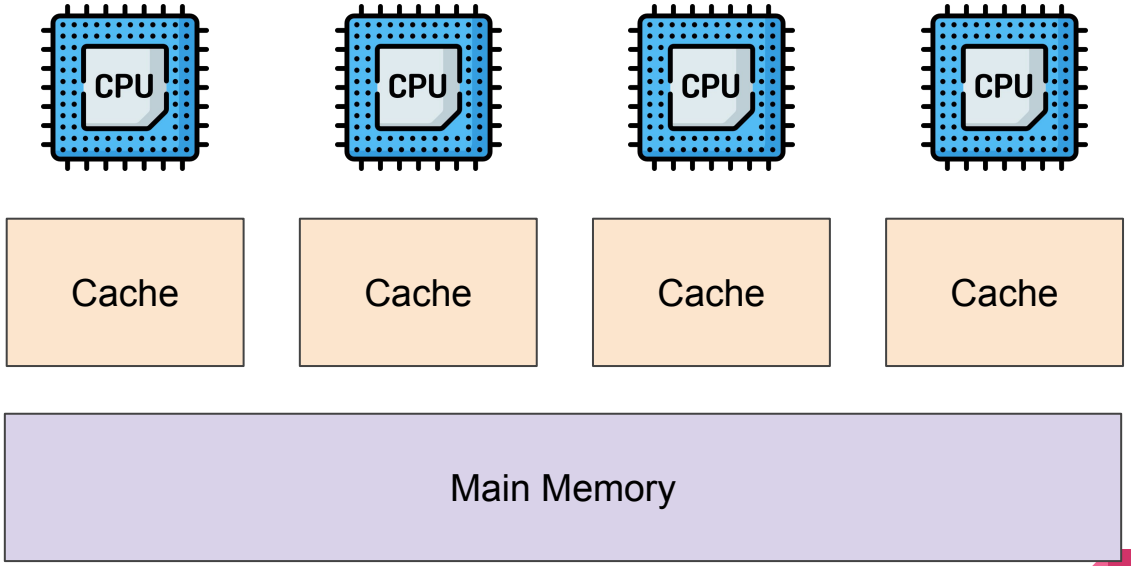




Ready Queue: Thread 2

Caches

Is it OK for a thread to move cores?

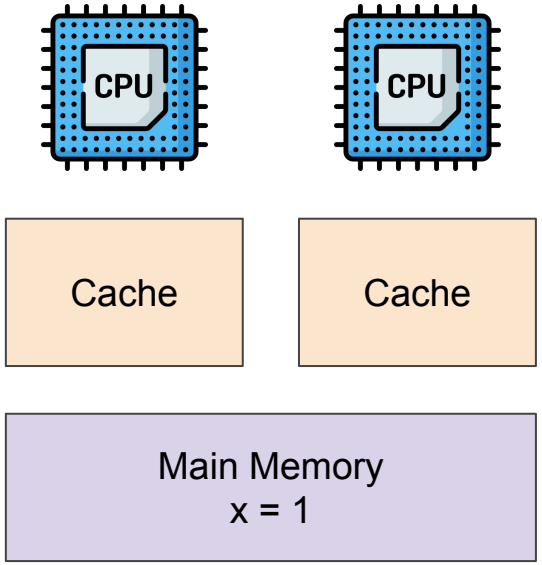


Caches

Is it OK for a thread to move cores?

Thread 0:

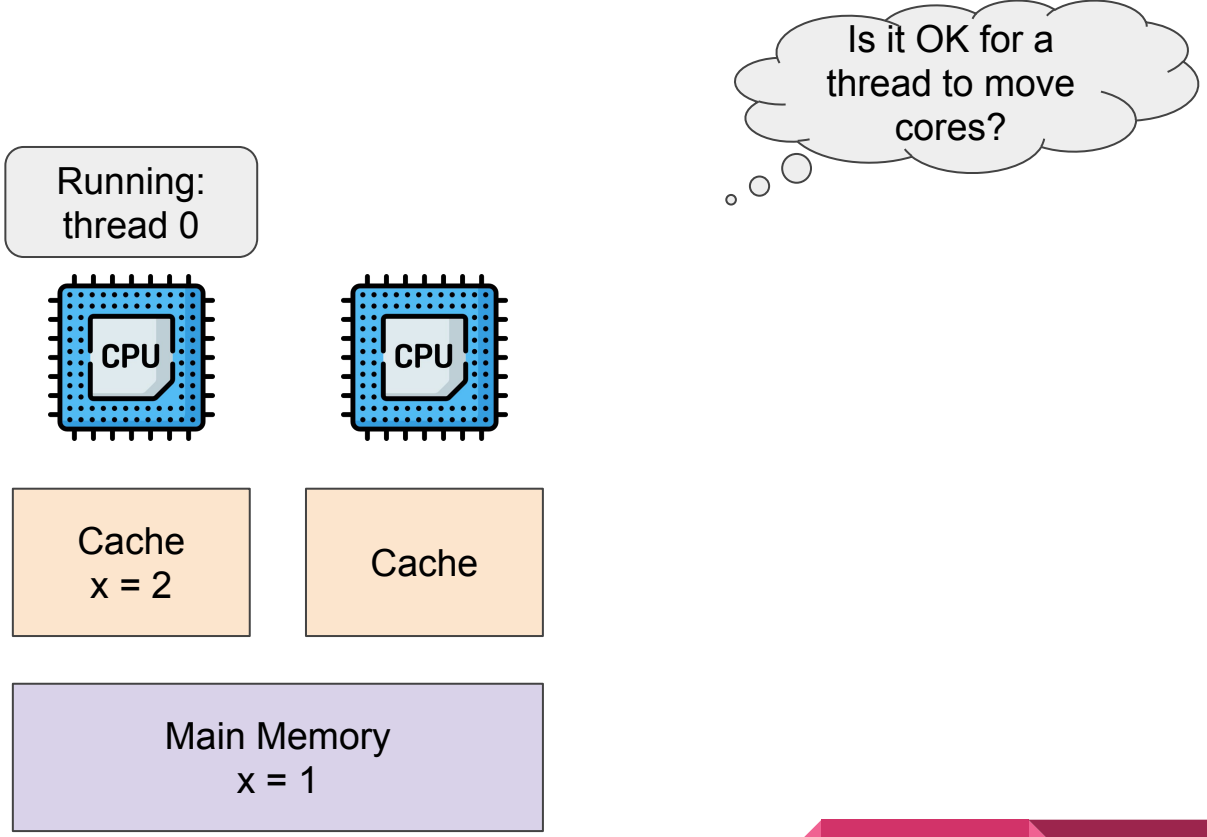
```
// x = 1 already

x++;
yield();
x++;

assert(x == 3);
```



Caches

Thread 0:

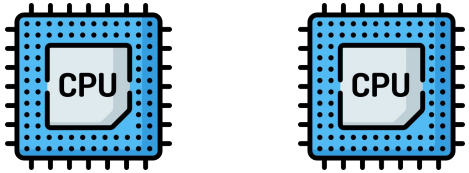
```
// x = 1 already

x++;
yield();
x++;

assert(x == 3);
```



Caches

Is it OK for a thread to move cores?

Running: thread 0



Cache
x = 2

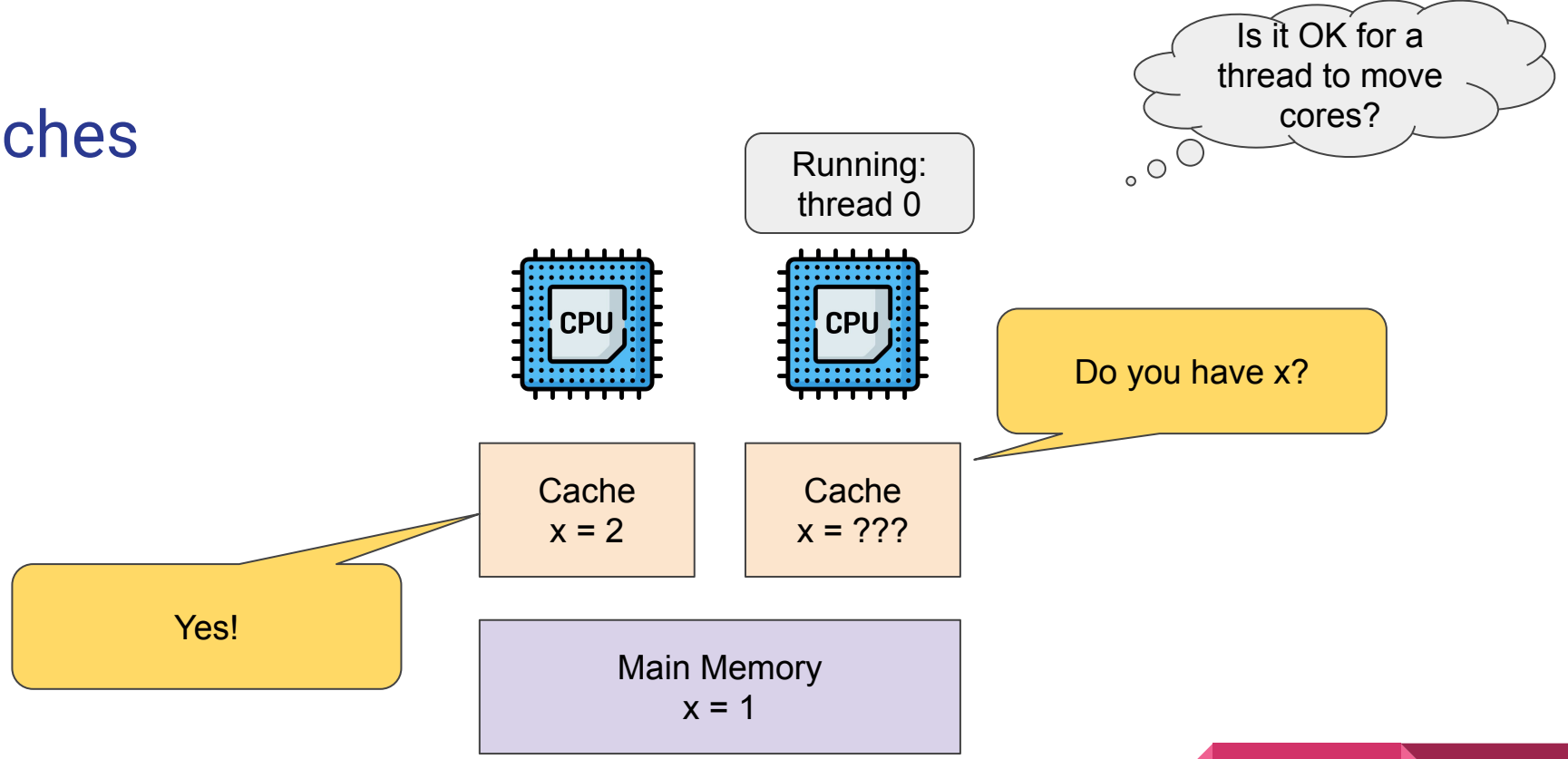
Cache
x = ???

Main Memory
x = 1

Thread 0:

```
// x = 1 already

x++;
yield();
x++;

assert(x == 3);
```

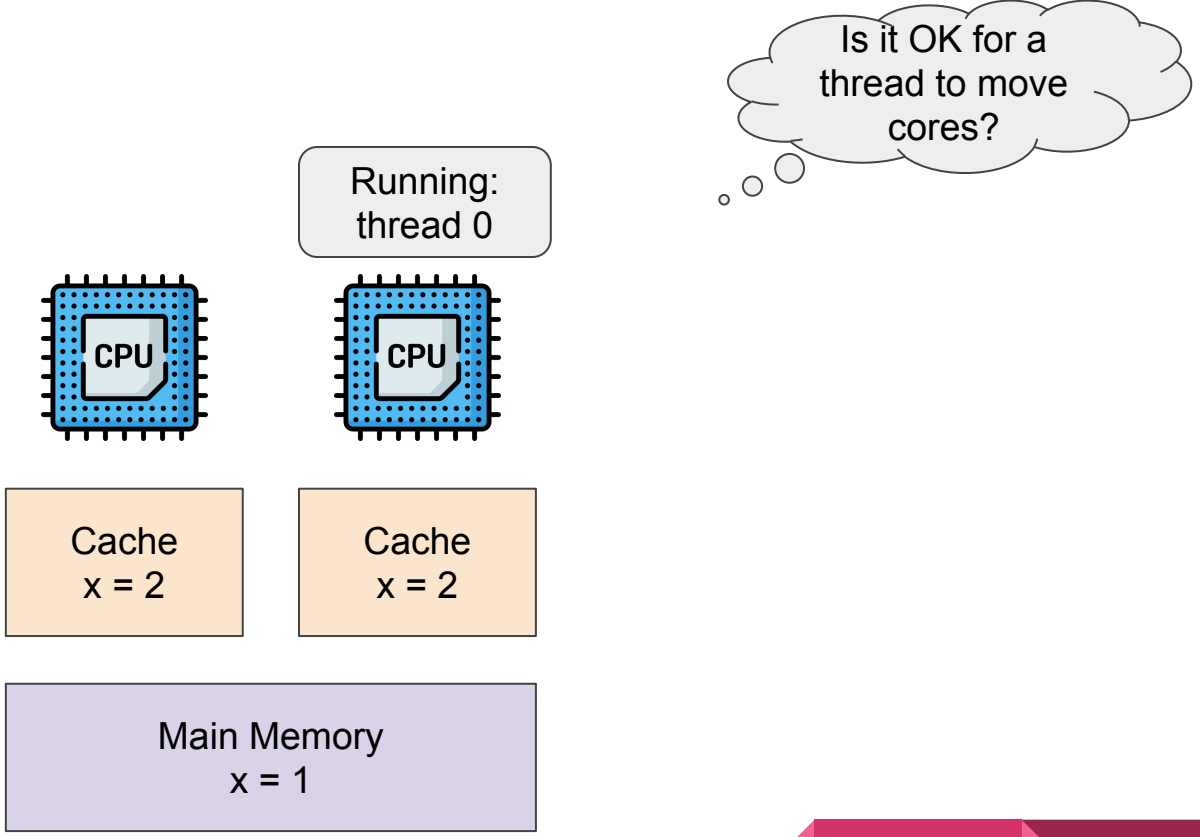
Caches


Is it OK for a thread to move cores?
Running: thread 0
CPU
CPU
Do you have x?
Cache
x = 2
Cache
x = ???
Yes!
Main Memory
x = 1

Caches

Thread 0:

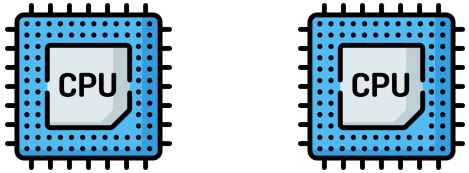
```
// x = 1 already

x++;
yield();
x++;

assert(x == 3);
```



Caches

Is it OK for a thread to move cores?

Running: thread 0



Cache
x = 2

Cache
x = 3

Main Memory
x = 1

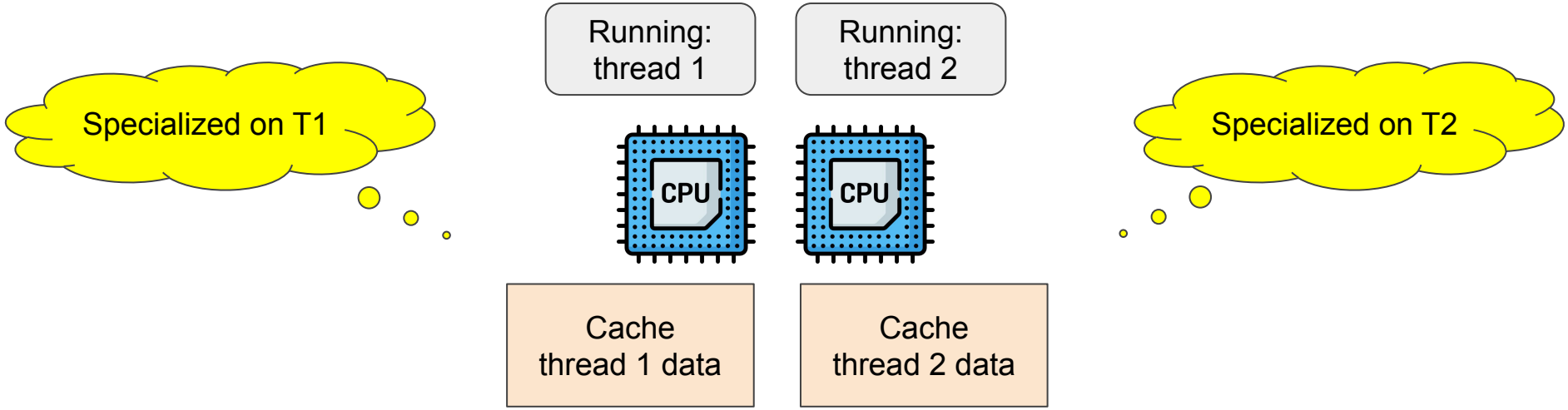
Thread 0:

```
// x = 1 already

x++;
yield();
x++;

assert(x == 3);
```

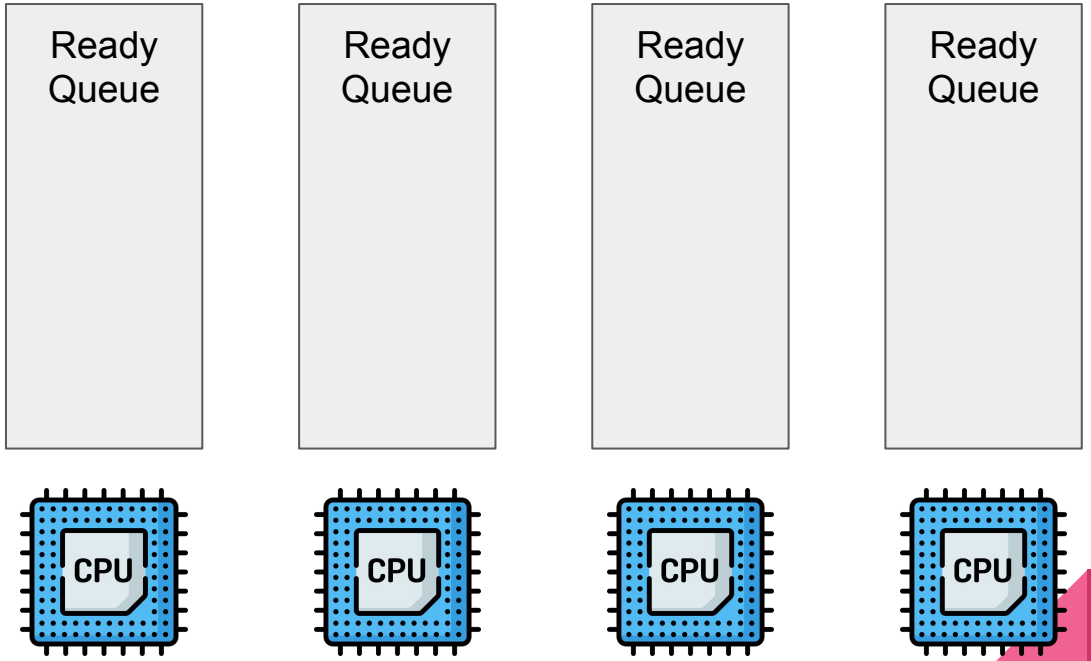
Takeaway

- Caches are completely invisible through a procedure called Cache Coherence
- Threads can move across cores without extra special programming effort
  - However, cache coherence traffic will slow down your program
  - In other words, staying on the same core is more performant

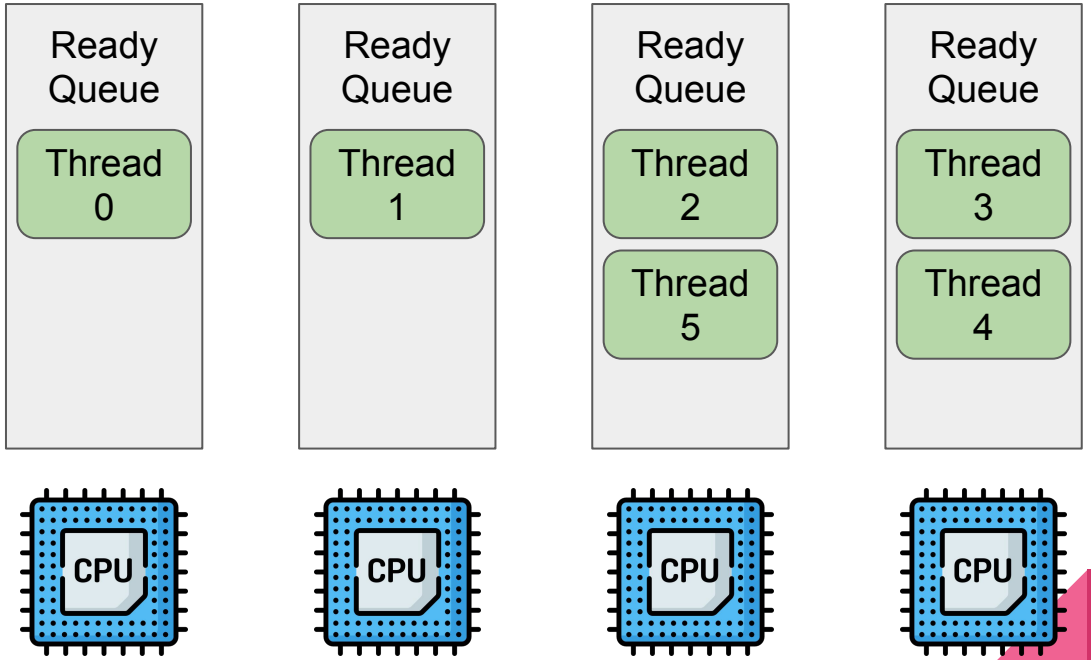
Core/Cache Specialization



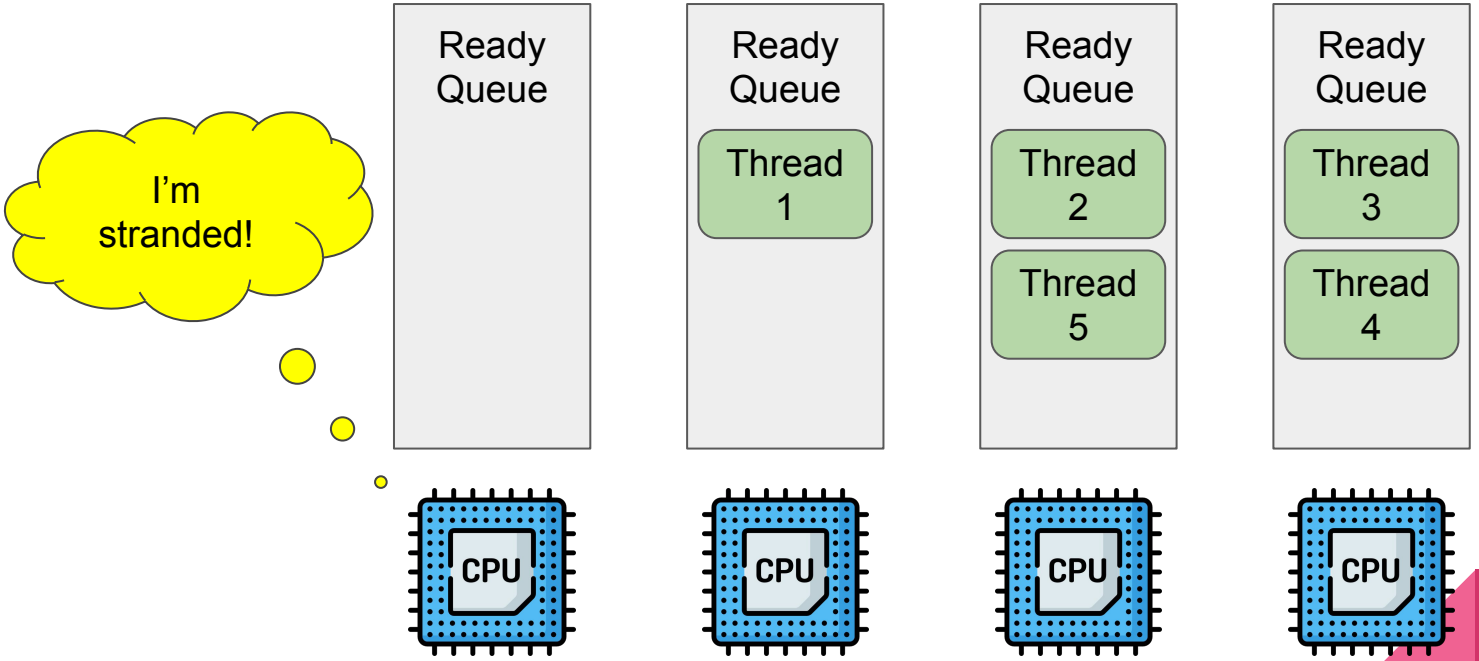
Core-Local Ready Queues



Core-Local Ready Queues



Problem: Core 0 has nothing to do!



Tradeoff

Specialization of cores vs. stranding cores

Scheduling

Which thread do we run on which core next?

- Open research problem in Computer Science
- You will talk more about it in OS

Tradeoffs: throughput vs. latency, specialization vs. stranding

Alert!

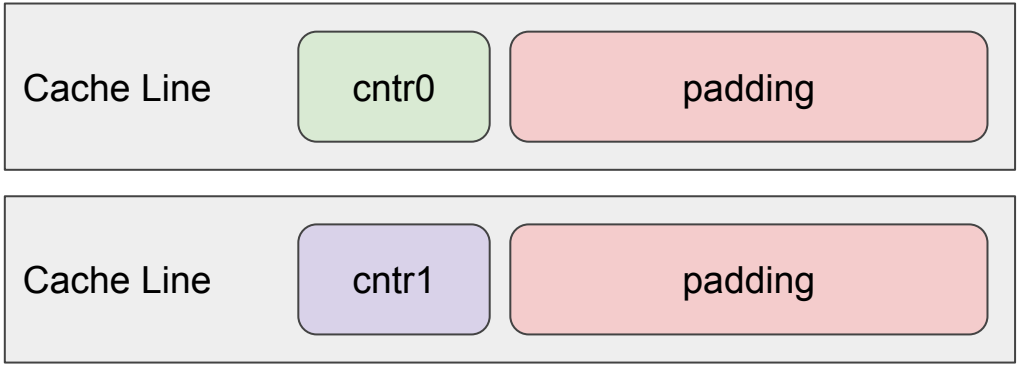
Important considerations about the real world ahead!

Cacheline Thrashing / False Sharing

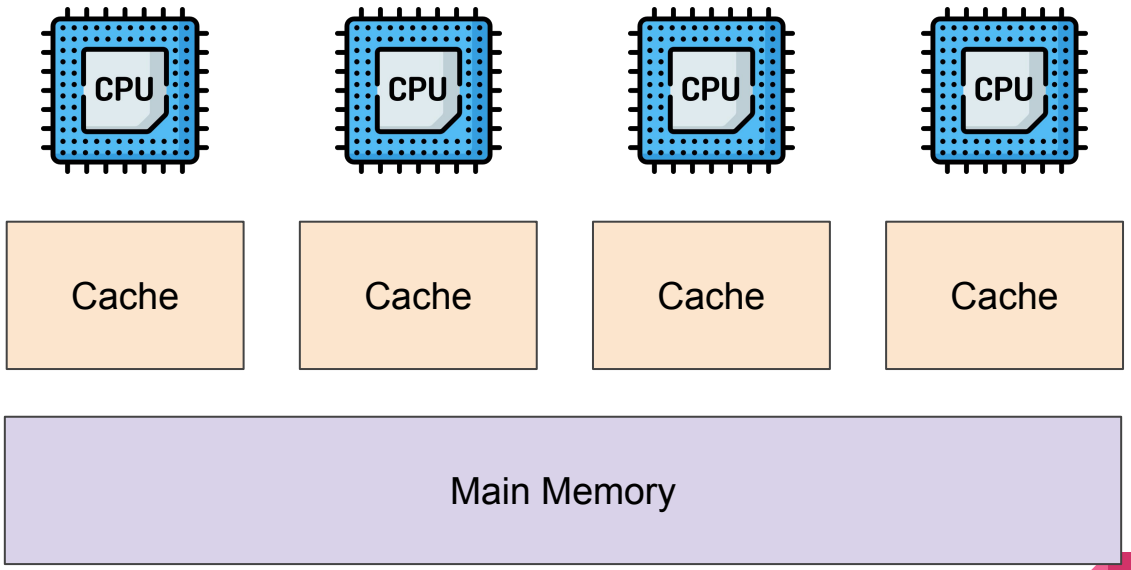
- Say core 0 is constantly incrementing cntr0, and core 1 with cntr1.
- If cntr0 and cntr1 are on the same cacheline / cache block, then the line will constantly bounce between the cores!
- Can cause massive performance degradation

False Sharing Solution: Cacheline Padding

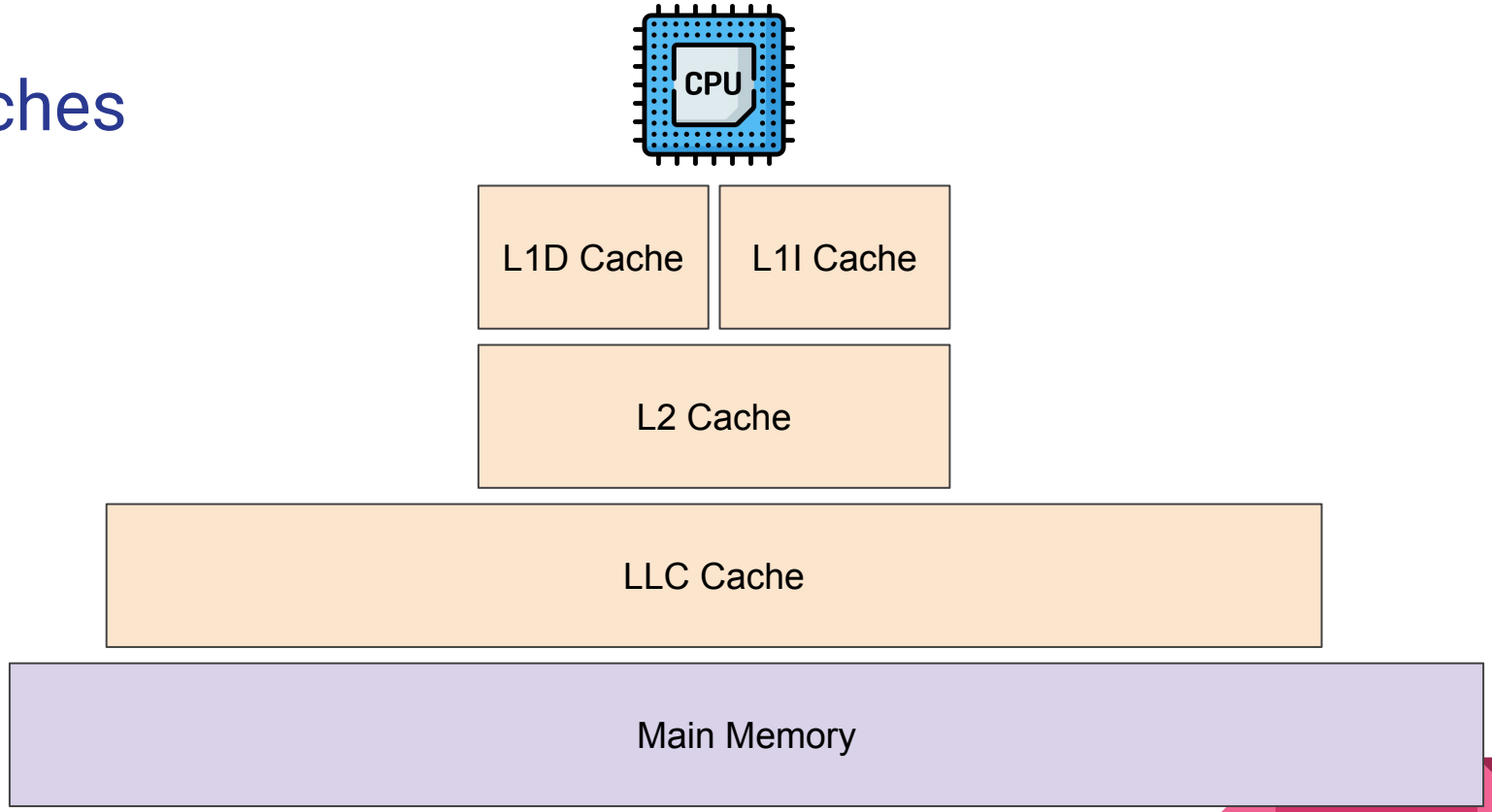
- To solve this, we must place cntr0 and cntr1 on different cachlines via padding!
- Wastes a ton of memory :(Another tradeoff!



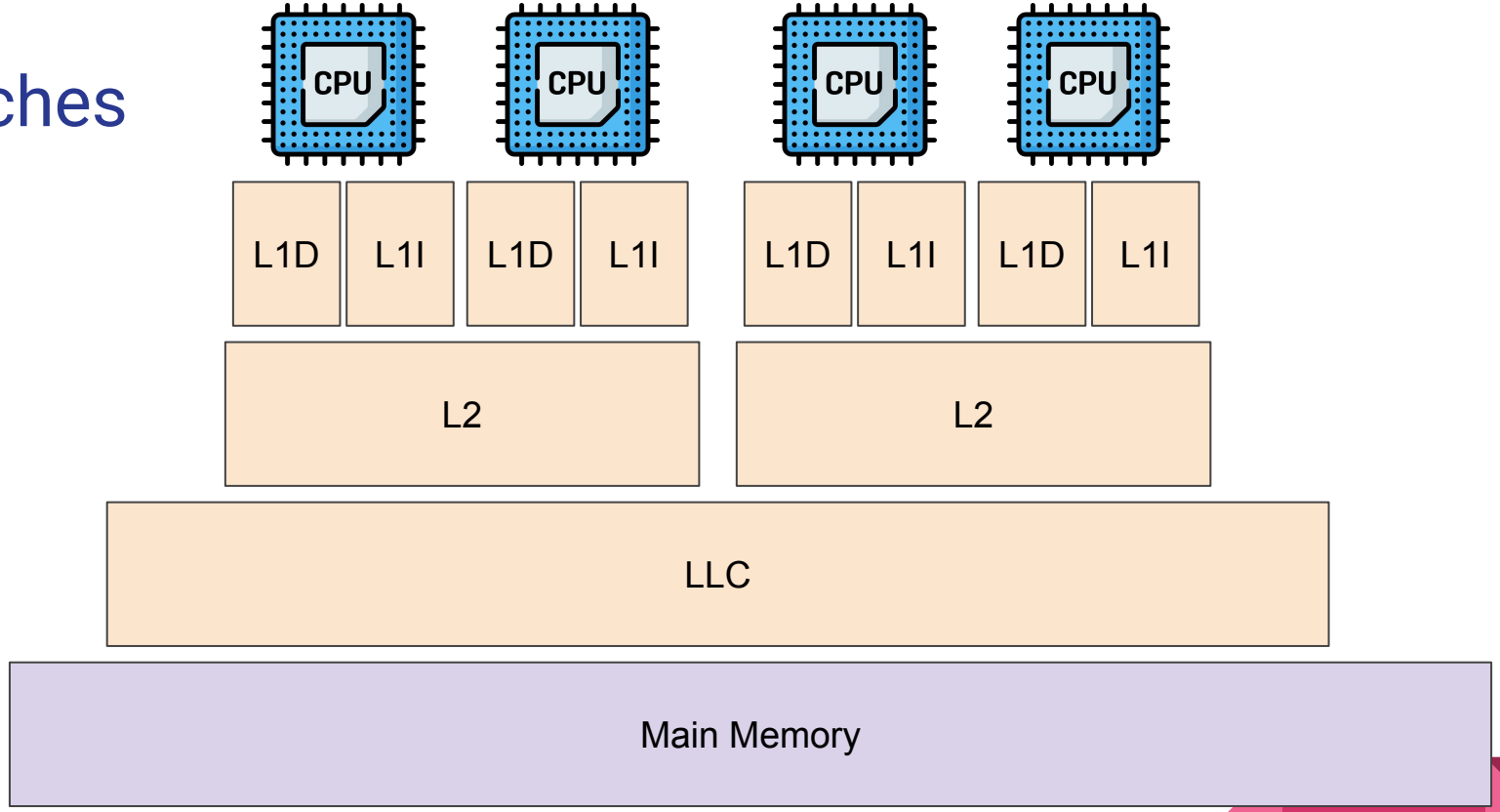
Caches



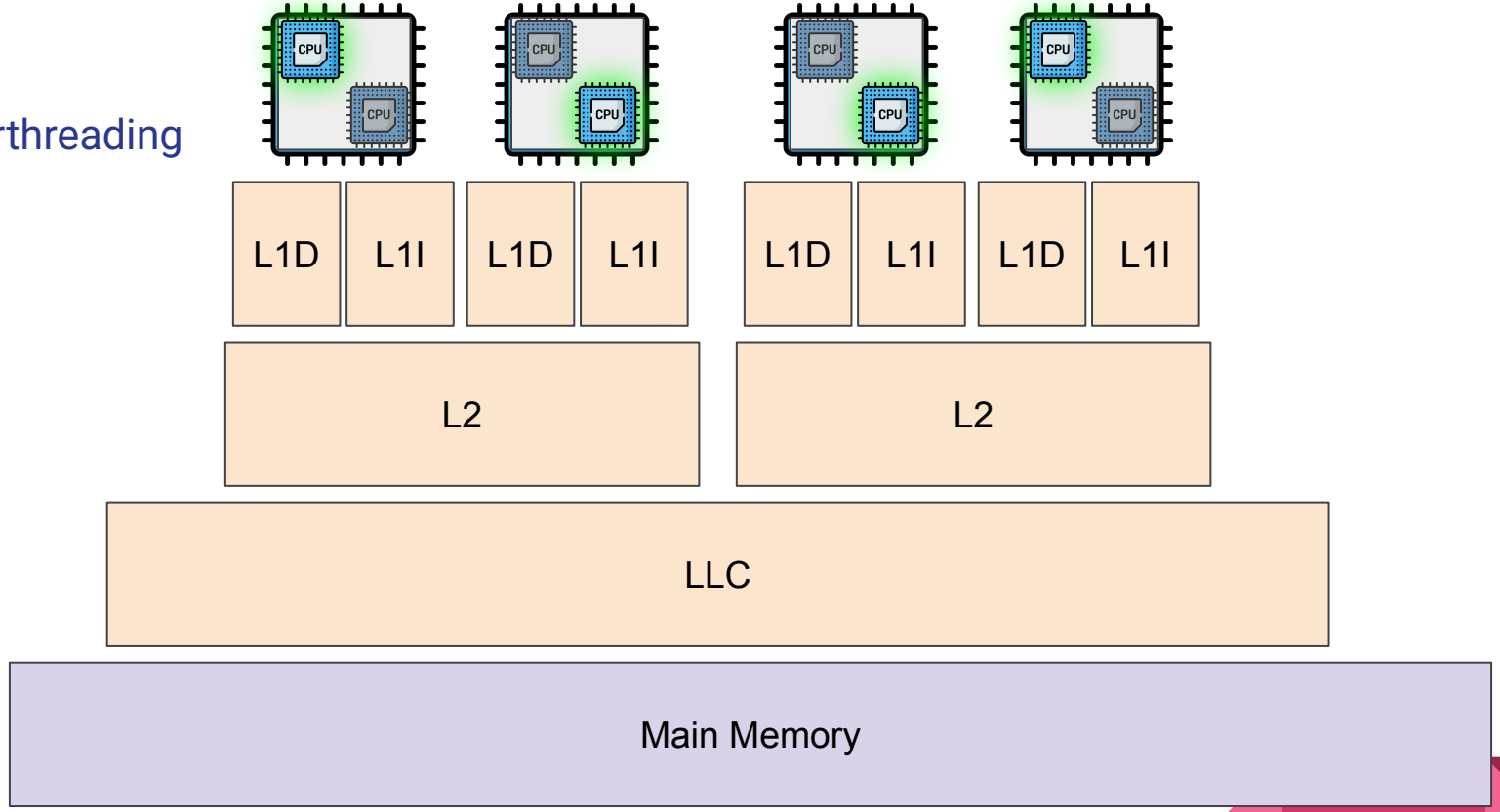
Caches



Caches


CPU
CPU
CPU
CPU
L1D
L1I
L1D
L1I
L1D
L1I
L1D
L1I
L2
L2
LLC
Main Memory

SMT/ Hyperthreading



Cores are complex

- They reorder instructions when possible and run multiple at once!

Core 0:

```
x++;
y++;
```

y might be in a register, whereas x requires going to memory, so y++ may finish first!

Cores are complex

- They reorder instructions when possible and run multiple at once!
- Normally not a problem with single-threaded code, but very problematic in parallel code
- This is known as “memory consistency”: When can a core reorder memory operations?

Compilers are complex

- They reorder instructions too!

Program:

```
x++;
y++;
```

y++ is placed FIRST in the ASM output because it has fewer dependencies

ASM:

```
y++;
x++;
```

Synchronization Primitives

Are mechanisms for synchronizing threads!

Guarantee that certain operations have finished

Barriers

Ensure that all threads have gotten to a certain point

```
void main():

        spawn(my_func)
        x = 5
```

```
void my_func():
        print(x)
```

Barriers

Ensure that all threads have gotten to a certain point

```
void main():
        barrier = new barrier
        spawn(my_func, barrier)
        x = 5
        barrier.wait()
```

```
void my_func(barrier):
        barrier.wait()
        print(x)
```

Barriers

Ensure that all threads have gotten to a certain point

```
void main():
        A = lots of data
        for 0..N:
                spawn(my_func, A)
        join(all threads)
```

```
void my_func(A):
        do_phase1(my portion of A)
        barrier.wait()
        do_phase2(my NEW portion of A)
```

Summary

Threads are a great abstraction

Sadly, we still need to know how cores work to build correct parallel code on threads, because threads run on cores

Synchronization primitives allow us to further abstract our thread code from how cores run, but performance suffers